Sentiment Analysis on Twitter Data

Peize Li
Technical University Berlin
Berlin Germany
peizeli@tu-berlin.de

Benedikt Konstantin Schäfer
Humboldt University of Berlin
Berlin Germany
benedikt.konstantin.schaefer@student.hu-berlin.de

1 Exploratory Data Analysis

1.1 Data Overview. The dataset contains 27,480 tweets after removing a single entry with missing text. Only the **text** and **sentiment** columns were retained for the sentiment classification task. Each tweet is labelled as either **negative**, **neutral**, or **positive**. The dataset was read using Latin-1 encoding.

1.2 Class Distribution. Figure 1 presents the sentiment distribution of the dataset. The neutral class is the largest, containing 11,117 tweets (40.5%), followed by the positive class with 8,582 tweets (31.2%) and the negative class with 7,781 tweets (28.3%).

The class distribution is moderately imbalanced. As a result, classification models may become biased toward the neutral class since predicting the majority class alone could already achieve approximately 40% accuracy. Furthermore, the neutral class represents an intermediate sentiment category and is therefore expected to be more difficult to distinguish from positive and negative tweets.

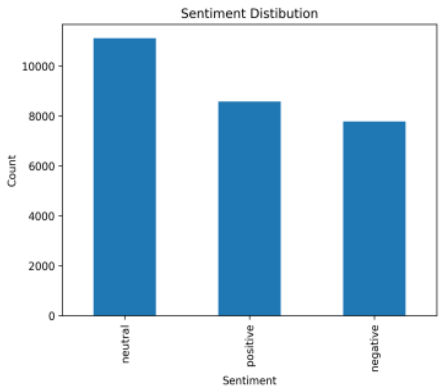


Figure 1. Distribution of sentiment classes in the dataset

1.3 Tweet Length Analysis. Tweets in the dataset are generally short, containing approximately 12–13 words and 65–70 characters on average. Negative tweets are slightly longer than neutral tweets, with average lengths of 13.5 and 12.3 words respectively.

The short length of tweets presents a challenge for sentiment classification because limited context is available. Consequently, Bag-of-Words and TF-IDF representations are expected to produce sparse feature vectors, where a small number of words can strongly influence classification decisions.

1.4 Vocabulary Analysis. Using simple whitespace tokenization, the dataset contains 52,270 unique tokens. Considering the relatively small dataset size, this indicates the presence of substantial lexical variation. Possible sources include Twitter-specific language, abbreviations, slang, usernames, spelling variations, and punctuation attached to words.

Such vocabulary noise may negatively affect classification performance because many tokens occur only once or twice. Therefore, preprocessing steps such as lowercasing, punctuation removal, and lemmatization are expected to reduce sparsity and improve feature quality.

1.5 Frequent Words and N-gram Analysis. After applying basic cleaning procedures, the most frequent words include *day*, *get*, *good*, *like*, *go*, *got*, *going*, *love*, *happy*, and *work*. While some words such as *good*, *love*, and *happy* clearly indicate positive sentiment, many high-frequency words are sentiment-neutral and therefore provide limited discriminative information.

Class-specific bigrams reveal more meaningful sentiment patterns. Positive tweets frequently contain expressions such as *good morning*, *good luck*, and *looking forward*. Negative tweets commonly contain phrases such as *sorry hear*, *going miss*, and *gonna miss*, while neutral tweets often contain time-related expressions such as *last night*, *next week*, and *getting ready*.

However, several bigrams appear across multiple sentiment classes. Examples include *wish could*, *feel like*, *looks like*, and *looking forward*. This overlap suggests that surface-level n-grams alone may not always be sufficient for distinguishing sentiment classes and that contextual interpretation remains important.



Figure 2. Word cloud showing the most frequent terms after basic text cleaning

1.6 Key Challenges. Several challenges are evident from the exploratory analysis:

- The neutral class is the largest class and may dominate model predictions.
- Tweets are short and contain limited contextual information.
- The vocabulary is noisy, containing many rare and Twitter-specific terms.
- Multiple n-grams appear across different sentiment classes, creating ambiguity.
- Sarcasm and implicit sentiment are not easily captured through lexical features alone.

These observations motivate the preprocessing decisions described in Task 2 and provide insight into the difficulties expected during sentiment classification.

2 Text Preprocessing

2.1 Data Cleaning. Prior to preprocessing, rows containing missing values in the text field were removed to prevent errors during subsequent processing stages and model training. After preprocessing, rows that became empty due to text cleaning operations were also discarded, as they contained no meaningful information for sentiment classification.

URLs were removed from tweet texts because they generally do not contribute to sentiment interpretation and may introduce noise into feature representations such as Bag-of-Words (BoW) and TF-IDF. Since most URLs follow common patterns beginning with "http" or "www", they can be identified and removed using regular expressions.

2.2 Text Normalization. All text was converted to lowercase to ensure that different capitalizations of the same word were treated consistently. This reduces vocabulary size and prevents words such as "Good" and "good" from being represented as separate features.

Punctuation and special symbols were removed to reduce noise in the textual representation. While certain punctuation marks may occasionally convey sentiment, tweets often contain excessive or informal punctuation usage that provides limited predictive value. All characters other than letters, numbers, underscores, whitespace, and asterisks were removed. Asterisks were retained because they frequently indicate censored swear words (e.g., "****"), which may carry negative sentiment. Numerical values were also preserved, as they may convey sentiment-related information such as ratings or scores.

2.3 Linguistic Processing. Lemmatization was applied to convert words to their canonical forms while preserving linguistic validity. Unlike stemming, which removes affixes through heuristic rules and may generate distorted word forms, lemmatization produces meaningful dictionary forms and therefore retains semantic information more effectively. Tokenization was performed as part of the lemmatization process to split each tweet into individual words before normalization.

Stop-word removal was also investigated to reduce the influence of highly frequent but semantically weak terms. However, negation words such as "not", "never", and "no" were excluded from the stop-word list because they play an important role in sentiment expression. Removing such terms may alter the meaning of a sentence and potentially reverse its sentiment polarity. Since we are not certain that we can cover all negation cases, we decided not to apply stop-word removal.

3 Sentiment Classification

3.1 Experiments Setup. The dataset was divided into training and testing sets using an 80/20 split. The experiments were conducted using two classification approaches: Multinomial Naive Bayes (NB) and a Feed-Forward Neural Network (FFNN). Both Bag-of-Words (BoW) and TF-IDF representations were evaluated. Model performance was assessed using accuracy, precision, recall, macro F1-score, and confusion matrices.

Although the dataset exhibited moderate class imbalance, the imbalance was not severe enough to justify mandatory resampling. Nevertheless, Random Oversampling was evaluated in several experiments to investigate its effect on classification performance.

3.2 Naïve Bayes Classification.

Representation	Configuration	Accuracy	Macro F1
TF-IDF Unigram	Default	0.60	0.58
TF-IDF Bigram + 5000 features	Best TF-IDF	0.64	0.64
BoW Unigram + 5000 features	Best NB	0.65	0.65

Table 1: Performance of Naïve Bayes Model

Multiple feature representations and preprocessing configurations were evaluated using the Multinomial Naive Bayes classifier. Overall, TF-IDF and BoW produced similar performance, with the best Naive Bayes configuration achieving an accuracy of 64.8% and a macro F1-score of 0.65 using a Bag-of-Words representation with a vocabulary size of 5,000 features.

Vocabulary size had a larger impact on performance than most preprocessing decisions. Restricting the vocabulary to the most frequent features reduced noise and improved classification accuracy. Conversely, increasing the vocabulary size to 50,000 features degraded performance, suggesting that many infrequent Twitter-specific terms introduced sparsity and reduced the effectiveness of the classifier.

The inclusion of bigram features did not consistently improve performance. Although bigrams can theoretically capture phrase-level sentiment, the short length and linguistic variability of

tweets caused many bigram features to occur infrequently. This increased sparsity and limited their usefulness for the Naive Bayes classifier.

Lemmatization produced minimal performance improvements, indicating that TF-IDF and BoW representations were already sufficiently robust to simple morphological variations. Similarly, adjusting the Naive Bayes smoothing parameter (alpha) resulted in only marginal changes in performance.

Random Oversampling improved recall for the minority sentiment classes but reduced overall accuracy. While the classifier became less biased toward the neutral class, the increase in false positive predictions outweighed the benefits of improved class balance.

3.3 Feed-Forward Neural Network Classification.

Configuration	Accuracy	Macro F1
100 neurons	0.68	0.69
200 neurons	0.67	0.67
(100, 50) layers	0.65	0.65
$\alpha = 0.01$	0.69	0.69
$\alpha = 0.1$	0.70	0.71

Table 2: Performance of Feed-Forward Neural Network

The Feed-Forward Neural Network consistently outperformed the Naive Bayes classifier across all experiments. The best-performing configuration used a Bag-of-Words representation with 5,000 features and a single hidden layer containing 100 neurons. By increasing the regularization parameter to alpha = 0.1, the model achieved approximately 70% accuracy and a macro F1-score of 0.71.

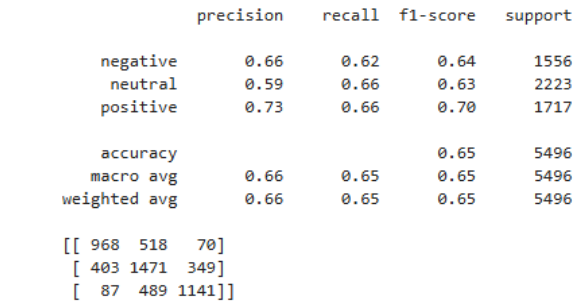
Unlike Naive Bayes, the neural network can learn interactions between input features rather than assuming that words occur independently. This capability enabled the model to better distinguish positive and negative sentiment, resulting in improved classification performance.

Several architectural modifications were explored. Increasing the number of training iterations from 10 to 50 reduced performance, suggesting overfitting to the sparse lexical features. Similarly, increasing the hidden layer size from 100 to 200 neurons did not improve results, indicating that the model was not constrained by insufficient network capacity. Introducing a second hidden layer also reduced performance, suggesting that additional complexity did not provide meaningful benefits for the Bag-of-Words representation.

The effect of regularization was more significant than changes in architecture. Increasing alpha from the default value to 0.1 improved generalization and produced the highest overall performance. This suggests that the network initially overfit to noisy lexical features and benefited from stronger regularization.

When TF-IDF was used instead of Bag-of-Words, performance decreased slightly. This finding indicates that the neural network was able to effectively exploit raw word-frequency information and did not benefit from the TF-IDF weighting scheme to the same extent as the Naive Bayes classifier.

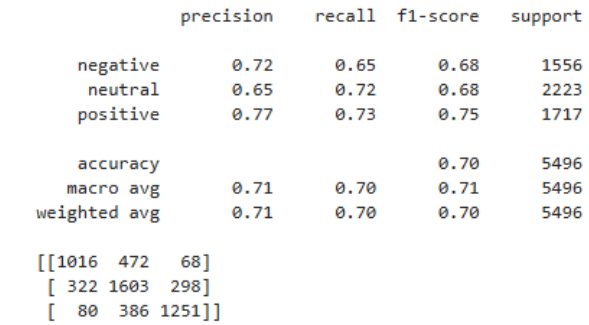
3.4 Error Analysis.



	precision	recall	f1-score	support
negative	0.66	0.62	0.64	1556
neutral	0.59	0.66	0.63	2223
positive	0.73	0.66	0.70	1717
accuracy			0.65	5496
macro avg	0.66	0.65	0.65	5496
weighted avg	0.66	0.65	0.65	5496

```
[[ 968  518   70]
 [ 403 1471  349]
 [  87  489 1141]]
```

Figure 3. Metrics and Confusion Matrix of Best Naïve Bayes Model



	precision	recall	f1-score	support
negative	0.72	0.65	0.68	1556
neutral	0.65	0.72	0.68	2223
positive	0.77	0.73	0.75	1717
accuracy			0.70	5496
macro avg	0.71	0.70	0.71	5496
weighted avg	0.71	0.70	0.70	5496

```
[[1016  472   68]
 [ 322 1603  298]
 [  80  386 1251]]
```

Figure 4. Metrics and Confusion Matrix of Best FFNN Model

Figures 3 and 4 show the confusion matrices of the best-performing Naive Bayes and FFNN models respectively.

Despite achieving reasonable performance, both classifiers exhibited recurring error patterns.

The most common error involved negative tweets being classified as neutral. Many of these tweets expressed sentiment indirectly through situations rather than explicit emotional vocabulary. For example, the tweet "cant go back to sleep got some work to do sigh sigh" was often classified as neutral because it lacked strongly negative lexical indicators despite clearly expressing frustration.

Similarly, positive tweets were frequently misclassified as neutral when sentiment was expressed subtly. An example is "yum do you do home delivery", where the positive sentiment is implied rather than stated directly through highly positive words such as "love" or "amazing".

Another source of error involved tweets containing mixed sentiment. For example, "shes smarter and classier shes perfect for him and im not" contains both admiration and self-deprecation. Such cases are difficult because the overall sentiment depends on understanding the relationship between multiple emotional cues.

Finally, some tweets required external context to determine sentiment correctly. For example, "about to go to my last theater lunch im gonna cry" may indicate sadness or happiness depending on the surrounding circumstances. Since neither classifier possesses external contextual knowledge, these tweets were frequently misclassified.

Overall, the majority of errors were associated with ambiguity, indirect sentiment expression, or insufficient contextual information rather than simple lexical mistakes.

3.5 Comparison Between Naïve Bayes and FFNN.

Model	Accuracy	Macro F1
Naïve Bayes	64.8%	0.65
FFNN	70.0%	0.71

Table 3: Performance of Naïve Bayes and FFNN

The Feed-Forward Neural Network achieved superior performance compared to the Naive Bayes classifier. The best FFNN configuration achieved approximately 70% accuracy, whereas the best Naive Bayes configuration achieved 64.8% accuracy.

Analysis of the confusion matrices revealed that both models primarily struggled with the neutral class. However, the FFNN produced fewer positive-to-negative and negative-to-positive confusions. This suggests that the neural network learned stronger sentiment boundaries and was better able to capture interactions between sentiment-bearing words.

The Naive Bayes classifier relies on the assumption that words are independent, which limits its ability to model complex relationships between features. In contrast, the neural network can learn nonlinear feature interactions, allowing it to capture richer sentiment patterns. Nevertheless, both approaches continued to struggle with tweets requiring contextual understanding or interpretation beyond lexical content.

3.6 Binary versus Multiclass Classification.

Model	Multiclass Accuracy	Binary Accuracy
Naïve Bayes	64.8%	87.0%
FFNN	70.0%	88.0%

Table 4: Performance of Naïve Bayes and FFNN on Multiclass and Binary

To further investigate classification difficulty, the neutral class was removed and the experiments were repeated as a binary sentiment classification task.

The best Naive Bayes model achieved 87% accuracy and a macro F1-score of 0.87 in the binary setting, compared to 64.8% accuracy and 0.65 macro F1-score in the multiclass setting. Similarly, the best Feed-Forward Neural Network achieved 88% accuracy and a macro F1-score of 0.88.

The substantial performance improvement demonstrates that the neutral class introduced significant ambiguity into the classification task. In the multiclass setting, many errors involved positive or negative tweets being assigned to the neutral category. Once the neutral class was removed, the models only needed to distinguish between positive and negative sentiment, resulting in a much simpler decision boundary.

Although the Feed-Forward Neural Network continued to outperform Naive Bayes in the binary setting, the performance gap became considerably smaller. This suggests that the additional modeling capacity of the neural network provides greater benefits when handling the more challenging multiclass classification problem.

4 Word Embeddings and Sentence Similarity

Fifteen tweets labelled as positive sentiment were selected from the dataset for semantic similarity analysis. Pre-trained GloVe-Twitter-100 embeddings were used to represent words as dense vectors, as these embeddings were trained on Twitter data and are therefore well suited to informal tweet language. Prior to vectorization, tweets were lowercased, punctuation and special characters were removed, and English stop words were filtered out. Stemming and lemmatization were not applied because the pre-trained embeddings already provide separate vector representations for different surface forms of a word. Across the selected tweets, only one out-of-vocabulary token was encountered.

Each sentence was represented by averaging the embeddings of its constituent words to obtain a sentence-level vector representation. Pairwise cosine similarities were then computed manually using the dot product and vector norms, without relying on pre-built similarity functions. Figure 5 presents the resulting cosine similarity matrix. Similarity values ranged from approximately 0.60 to 0.94, indicating that most sentences shared a moderate degree of semantic similarity. The most similar sentence pair (indices 3 and 11) achieved a cosine similarity of 0.944, despite discussing different topics. This suggests that semantically related words such as *glad*, *nice*, *cool*, and *love* occupy nearby positions in the embedding space. In contrast, the least similar pair (indices 1 and 9) obtained a similarity score of 0.596 due to minimal vocabulary overlap, differing topics, and the presence of short slang-heavy text.

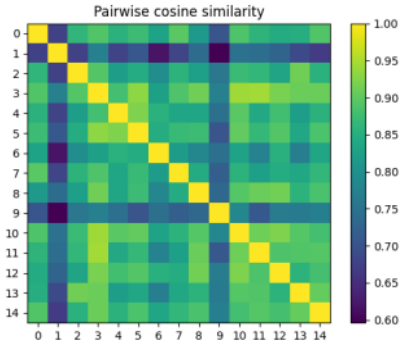


Figure 5. Pairwise cosine similarity matrix generated using averaged GloVe-Twitter-100 sentence embeddings.

These results demonstrate that word embeddings are capable of capturing semantic relationships beyond exact word matching. However, the averaging approach ignores word order and contextual information, meaning that sentences with different meanings may still obtain similar representations. Furthermore, very short tweets are represented by only a few word vectors, making their sentence embeddings more sensitive to noise and less reliable. Since all selected examples were positive tweets, baseline similarity values also remained relatively high, limiting the ability of the method to strongly separate sentence pairs.

5 Pre-trained Language Models

We fine-tuned DistilBERT (distilbert-base-uncased) for sentiment classification in both multiclass and binary settings. Unlike the classical models, the transformer was trained on the raw tweet text, allowing it to exploit contextual information, punctuation, and word order. Fine-tuning used a learning rate of 2e-5, batch size 16, three epochs, and weight decay 0.01. These settings were chosen to adapt the model while limiting overfitting.

Table 5 compares DistilBERT with the best Naive Bayes and FFNN models. DistilBERT achieved 0.79 accuracy and 0.80 macro F1-score in the multiclass setting, outperforming the best FFNN (0.71 macro F1) and Naive Bayes (0.65 macro F1). In the binary setting, performance increased further to 0.94 accuracy and 0.94 macro F1-score. The smaller gap between transformer and classical models in the binary task suggests that distinguishing the neutral class is the main source of difficulty.

Model	Setting	Accuracy	Macro-F1
Naïve Bayes (best)	Multiclass	65%	0.65
FFNN (best)	Multiclass	70%	0.71
DistilBERT	Multiclass	79%	0.80
Naïve bayes (best)	Binary	87%	0.87
FFNN (best)	Binary	88%	0.88
DistilBERT	Binary	94%	0.94

Table 5. Performance comparison of Naive Bayes, FFNN, and DistilBERT in multiclass and binary settings.

Error analysis and figure 6 showed that most remaining mistakes involved the neutral class. Common errors included indirect sentiment, mixed sentiment, context-dependent expressions, and ambiguous labels. Compared with Naive Bayes and FFNN, DistilBERT produced fewer cases where positive or negative tweets were incorrectly classified as neutral, demonstrating the benefit of contextual language understanding.

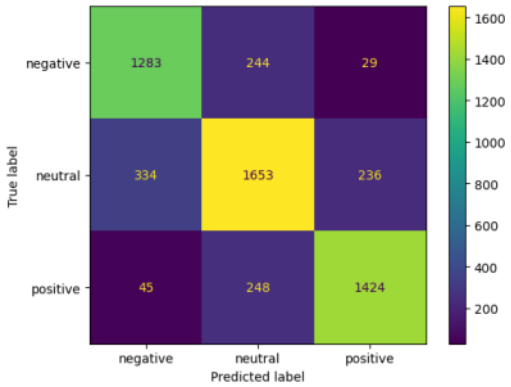


Figure 6. Confusion matrix of DistilBERT in the multiclass setting

Although DistilBERT achieved the best overall performance, it required substantially higher computational cost. Training required approximately 23 minutes for the multiclass task and 14 minutes for the binary task, whereas the classical models trained within seconds. In addition to longer training times, inference is also slower than the classical models due to the larger number of parameters. Therefore, the transformer model provides a clear accuracy advantage, particularly for context-dependent tweets, at the expense of efficiency.

REFERENCES

[1] Steven Bird, Ewan Klein, and Edward Loper. 2009. *Natural Language Processing with Python*. O'Reilly Media, Sebastopol, CA.

[2] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake VanderPlas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. 2011. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research* 12 (2011), 2825–2830.

[3] Guillaume Lemaitre, Fernando Nogueira, Christos K. Aridas, et al. 2017. *imbalanced-learn: A Python Toolbox to Tackle the Curse of Imbalanced Datasets in Machine Learning*. Journal of Machine Learning Research 18, 17 (2017), 1–5.

[4] Jeffrey Pennington, Richard Socher, and Christopher D. Manning. 2014. GloVe: Global Vectors for Word Representation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, Doha, Qatar, 1532–1543.

[5] Radim Řehůřek and Petr Sojka. 2010. Software Framework for Topic Modelling with Large Corpora. In *Proceedings of the LREC 2010 Workshop on New Challenges for NLP Frameworks*. ELRA, Valletta, Malta, 45–50.

[6] Andreas Müller. 2024. *wordcloud: A Little Word Cloud Generator in Python*. Retrieved from https://github.com/amueller/word_cloud

[7] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, and Jamie Brew. 2020. Transformers: State-of-the-Art Natural Language Processing. In Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations. Association for Computational Linguistics, 38–45.

[8] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. 2019. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. arXiv:1910.01108.

[9] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In Proceedings of NAACL-HLT 2019. Association for Computational Linguistics, 4171–4186.

[10] Lhoest, Quentin, et al. 2021. Datasets: A Community Library for Natural Language Processing. In Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing: System Demonstrations. Association for Computational Linguistics, 175–184..